

The Installed Scripts

The thirty-seven PowerShell scripts installed beside SD - the execution policy they need, what their exit codes mean, and the ones you may need to run yourself.

SD's installer does most of its work in PowerShell rather than inside the installer script, and **it leaves every one of those scripts on the machine**. They are in `C:\Program Files\SD`, beside `sd.exe`, and they are there for two reasons: so that a step which failed during the installation can be run again without reinstalling, and so that a choice made in the wizard can be changed afterwards.

They are Windows scripts, not SD verbs. Nothing here is typed at an `sd` prompt. Run them from a PowerShell prompt started with **Run as administrator** - almost all of them refuse a prompt that is not elevated, and say so rather than half-working. `check-install.ps1` **is the exception and wants an ordinary prompt**; the section on it says why.

Italics mark something you supply, **bold** a word typed as it stands, and braces an optional part.

What is here and what is not

Thirty-seven scripts ship. They are the installer's own steps, the helpers SD launches while it is running, and the ones the administrator verbs call. Everything else in the project's `gplbld` directory - the verifiers, the probes, the build and test cycle - is development tooling and **is deliberately not installed**. If you have read about `cycle.ps1`, `assert-current.ps1` or a `verify-` script and cannot find it, that is why: they compare an install against the source tree it was built from, and they are destructive.

PowerShell execution policy — you do not need to change it

Windows will not run a PowerShell script unless its **execution policy** allows it. On Windows desktop editions the default is `Restricted`, which allows no script at all; on Windows Server it is `RemoteSigned`, which allows a script written on the machine itself.

SD does not depend on that setting, and you should leave it alone. Every one of the thirty-seven scripts is launched with an explicit `-ExecutionPolicy Bypass` on its own command line — by the installer, by the SD service, and by the SD verbs that call one. That switch applies to **that one PowerShell process, for that one script**. It changes nothing on the machine and nothing about any other script.

What happens if you change it anyway:

Tighten it to `Restricted` or `AllSigned`, for the machine or for your own account

Nothing. SD carries on working. The switch SD passes takes precedence over both of those settings

Loosen it to `RemoteSigned`, `Unrestricted` or `Bypass`

Nothing — and you have gained nothing. SD was already unaffected. You have made the machine more permissive for every *other* script on it, which is a real cost for no benefit

Set it through Group Policy

This one stops SD. See below

So if you loosened the policy to get SD working, you can put it back. That was needed on builds before 5 September 2026, where SD's own commands failed with *"running scripts is disabled on this system"*. It is fixed, and the workaround is no longer doing anything for you.

One thing it does not cover. The `sh` verb opens an ordinary PowerShell prompt for you, and that prompt gets **no** such switch — it runs under whatever policy your machine sets. That is deliberate: SD lifts the restriction for the scripts it installed itself, and never for a shell you type into.

GROUP POLICY IS THE EXCEPTION, AND IT IS THE ONE TO KNOW ABOUT

A **Group Policy** setting — *Turn on Script Execution*, under `Computer Configuration` or `User Configuration` → `Administrative Templates` → `Windows Components` → `Windows PowerShell` — **outranks the switch SD passes**. Group Policy sits above the per-process setting in PowerShell's order of precedence, so on a machine where a policy sets the execution policy, SD cannot override it.

On a domain-joined or otherwise managed machine, check this before installing. In an ordinary PowerShell prompt:

```
Get-ExecutionPolicy -List
```

If the `MachinePolicy` or `UserPolicy` row says anything other than `Undefined`, a policy is in force. `RemoteSigned`, `Unrestricted` or `Bypass` **there is fine** — SD's scripts are written on the machine by the installer, not downloaded. **A policy of `Restricted` or `AllSigned` will stop SD's administrative commands**, and the symptom is the *"running*

scripts is disabled on this system" message from `logto sdsys`, `append.sd.path`, `remote.api`, `remote.ssh` or an editor verb. **That needs your Windows administrator to relax the policy.** SD has no way around it, deliberately: a program that could defeat Group Policy would be a worse thing to have installed than an inconvenience.

The exit codes are a convention

Every script prints what it did and then exits on the same three-value convention:

-
- 0 it is done. That includes *"it was already done"* - the scripts are written to be run twice
 - 1 it failed, and the line above the exit says why
 - 2 **it neither did the work nor failed.** It refused, or it could not run, or the work needs a restart first
-

2 Is the one worth reading. It is not an error code; it means the script declined to act and is telling you the condition. Each script below says what its own 2 means, because they differ - a refusal in `allow-ssh-groups.ps1` is not the same event as a pending restart in `install-ssh.ps1`. Only `sd-elevate.ps1` adds a fourth, **5**, for *not elevated*.

The ones you may need to run

These answer a question or change a decision. Every command below is complete as written; run it from an elevated PowerShell prompt.

IS THE INSTALLATION SOUND?

```
powershell -File "C:\Program Files\SD\check-install.ps1"
powershell -File "C:\Program Files\SD\check-install.ps1" -Brief
```

Exit **0** nothing is wrong, **1** something is. `-Brief` prints one line per check with no preamble. **A check that cannot be answered yet is not a failure** and it says so separately.

Run this one without elevation. It is the only script here that does, and the reason is the question it asks: *can this user's ordinary sign-in reach SD?* An administrator token reads the data tree through the `Administrators` entry on the ACL and would pass whether or not the answer is yes. The script notices it is elevated and says what the answer is worth, but that is a backstop rather than the intent. The Start Menu entry **Check the SD installation** runs it exactly this way.

And run it again later. The installer offers this check as a tick box at the end, and that run is **always the incomplete one**: the installing user's logon token cannot carry the `sdusers` group until they sign out and back in, so every database check reports *"not yet"* by design. The Start Menu entry is there so the real run can be done afterwards without writing the command down.

THE SSH SERVER WOULD NOT INSTALL

```
powershell -File "C:\Program Files\SD\install-ssh.ps1"
```

Exit **0** installed and running, **2** installed but Windows needs a restart before the service exists, **1** failed.

This is the one that matters most. Accounts SD creates sign in over ssh and nothing else, so until this succeeds nobody but you can use that SD. The installer prints this same command in its closing report when it could not install the server; it is repeated here because that report is easy to close. **It is slow** - `Add-WindowsCapability` downloads from Windows Update and can work for minutes in silence. Do not interrupt it.

AN EDITOR VERB DOES NOTHING

```
powershell -File "C:\Program Files\SD\install-editors.ps1"  
powershell -File "C:\Program Files\SD\install-editors.ps1" -CheckOnly
```

Exit **0** every editor is present, **2** at least one is missing and could not be installed, **1** failed. `-CheckOnly` reports and installs nothing. What happened last time is in `C:\ProgramData\SD\install-editors.log`.

THE SD SERVICE

```
powershell -File "C:\Program Files\SD\install-service.ps1" -Install  
powershell -File "C:\Program Files\SD\install-service.ps1" -Remove
```

Exit **0** done, **1** failed, **2** could not be attempted - not elevated, or `sdsvc.exe` is not there. `-Install` accepts `{-AppDir directory}` if SD is not in the usual place.

WHO MAY REACH THE API FROM OTHER COMPUTERS

```
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Show  
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Open  
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Restrict  
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Remove
```

Exit **0** applied, **1** failed, **2** refused. `-Show` changes nothing. `-Open` allows any address, `-Restrict` this machine only; add `{-Port n}` for a port other than 4243. **This script owns its rule** - it created it, and `-Remove` takes it away.

WHO MAY REACH SSH FROM OTHER COMPUTERS

```
powershell -File "C:\Program Files\SD\ssh-firewall.ps1" -Show  
powershell -File "C:\Program Files\SD\ssh-firewall.ps1" -Installed -Restrict  
powershell -File "C:\Program Files\SD\ssh-firewall.ps1" -Installed -Open
```

Exit **0** applied, **1** failed, **2** refused, or the rule is not there yet.

It toggles a rule it did not create. Installing the OpenSSH capability creates `OpenSSH-Server-In-TCP` and enables it for any address; this narrows it to loopback or widens it again. It has no `-Remove`, deliberately: the rule is Microsoft's and SD must not delete it.

WHO MAY SSH INTO THIS MACHINE AT ALL

```
powershell -File "C:\Program Files\SD\allow-ssh-groups.ps1" -Check
powershell -File "C:\Program Files\SD\allow-ssh-groups.ps1" -Installed
powershell -File "C:\Program Files\SD\allow-ssh-groups.ps1" -Remove
```

Exit **0** done or nothing to do, **1** failed, **2** refused.

The write needs `-Installed` and the script's own Usage text leaves it out. Without it you get *"-Installed not given"* and exit 2, and nothing is written. The switch means *an administrator asked for this*: the script rewrites `sshd_config` and restarts sshd, so it will not do that merely because it was run. `-Check` and `-Remove` do not need it.

It also refuses if `sshd_config` already says who may connect - an existing `AllowGroups`, `AllowUsers`, `DenyGroups` or `DenyUsers` line is somebody else's decision and is left alone. That refusal is also exit 2, and it prints the lines it found.

THE TWO REMOTE-ROUTE GROUPS

```
powershell -File "C:\Program Files\SD\sync-route-groups.ps1" -Check
powershell -File "C:\Program Files\SD\sync-route-groups.ps1"
```

Exit **0** success, **1** failure; it prints what it did either way. It creates the groups that decide which remote route an SD account may use, and seeds `sdssh` so an install that predates them does not lose ssh. `-Check` prints what it would do and changes nothing.

RE-STAMPING THE ACCOUNT DIRECTORIES

```
powershell -File "C:\Program Files\SD\secure-account-dirs.ps1" -Root "C:\ProgramData\SD\user_a
powershell -File "C:\Program Files\SD\secure-account-dirs.ps1" -Root "C:\ProgramData\SD\user_a
```

Exit **0** every directory stamped, **1** at least one failed, **2** it could not run. Add *{-Account name}* for one account rather than all of them.

This is the only `secure-` script with a reason to be run again. The others name one fixed path and the installer has already done them; this one walks a directory whose contents grow as accounts are made, and `create.account` stamps each new account itself.

MAY SD INSTALL ON THIS MACHINE?

```
powershell -File "C:\Program Files\SD\ssh-preflight.ps1"
```

Exit **0** clear to install, **1** refuse, **2** could not determine - which is also a refusal. **It changes nothing:** it reads the service registry, one TCP table and two files. The installer runs it before the wizard is drawn, and running it yourself is how you find out in advance why an installation would be refused on a machine that already has an ssh server.

Continued in

The Scripts SD Runs For Itself — the scripts the installer, the verbs, the uninstaller and SD itself run.
